1 . can i run this in linux ? or only windows ?

2. do i need to install all python/flex dependecies before docker image run ?
3.if so make me a bat fie to install all dependencies for this program to run python and flex

Here are clear answers to all 3 questions:

Q1 — Can you run this on Linux?

Yes, absolutely — and Linux is actually better for Docker. [1] [2]

Platform	Docker Support	BAT File	Use Instead
Windows 10/11	✅ Docker Desktop	✅ `.bat` works natively	—
Linux (Ubuntu/Debian)	✅ Docker Engine (lighter, faster)	❌ `.bat` doesn't run	Use `.sh` shell script
macOS	✅ Docker Desktop	❌ `.bat` doesn't run	Use `.sh` shell script

On Linux you use **Docker Engine** (not Desktop) — it's lighter, runs as a daemon, and is production-standard. `.bat` files are **Windows-only** — on Linux you need a `.sh` shell script instead. [3] [4] [5]

Q2 — Do you need to install Python/Flask before Docker runs?

No — that is exactly the point of Docker. The `Dockerfile` already installs everything inside the container:

- Python 3.11
- Flask, oracledb, Pillow, Gunicorn
- All system libraries (libjpeg, etc.)

On the target machine you only need **Docker itself** — nothing else. No Python, no venv, no pip.

Q3 — Two deploy scripts: one for Windows, one for Linux

☒ `install_and_run.bat` — Windows (updated with pip deps)

```
@echo off
title NU Tabulation Archive - Full Setup
color 0B
cls

echo.
echo ═══════════════════════════════════════════════════════════════
echo  NU TABULATION ARCHIVE  -  Full Windows Setup
echo  Installs: Python + pip deps  OR  Docker (your choice)
echo ═══════════════════════════════════════════════════════════════
echo.

set PROJECT_DIR=%~dp0
cd /d "%PROJECT_DIR%"

echo Choose setup mode:
echo   [^1] Run with Python directly (install pip dependencies)
echo   [^2] Run with Docker (recommended for deployment)
echo.
set /p CHOICE="Enter 1 or 2: "

if "%CHOICE%"=="1" goto :python_mode
if "%CHOICE%"=="2" goto :docker_mode
echo Invalid choice.
pause
exit /b 1

:: ═══════════════════════════════════════════════════════════════════
:python_mode
:: ═══════════════════════════════════════════════════════════════════
echo.
echo [MODE] Python direct mode selected.
echo.

:: ── Check Python ──────────────────────────────────────────────
echo [1/4] Checking Python installation...
python --version >nul 2>&1
if %errorlevel% NEQ 0 (
    echo [!] Python not found. Installing via winget...
    winget install -e --id Python.Python.3.11 --silent --accept-package-agreements --acce
    if %errorlevel% NEQ 0 (
        echo [!] winget failed. Downloading Python 3.11 installer...
        curl -L "https://www.python.org/ftp/python/3.11.9/python-3.11.9-amd64.exe" -o "%T
        "%TEMP%\python_installer.exe" /quiet InstallAllUsers=1 PrependPath=1
        echo [OK] Python installed. Refreshing PATH...
        set "PATH=%PATH%;C:\Python311;C:\Python311\Scripts"
    )
    echo [OK] Python installed.
) else (
    python --version
    echo [OK] Python found.
)

:: ── Create venv ───────────────────────────────────────────────
```

```
echo.
echo [2/4] Setting up virtual environment...
if not exist "%PROJECT_DIR%venv\Scripts\activate.bat" (
    python -m venv "%PROJECT_DIR%venv"
    echo [OK] Virtual environment created.
) else (
    echo [OK] Virtual environment already exists.
)

call "%PROJECT_DIR%venv\Scripts\activate.bat"

:: ── Install dependencies ──────────────────────────────────────
echo.
echo [3/4] Installing Python dependencies...
python -m pip install --upgrade pip

pip install flask==3.1.3
pip install oracledb==3.4.2
pip install pillow==12.1.1
pip install werkzeug==3.1.5
pip install cryptography==46.0.5
pip install gunicorn==23.0.0

echo.
echo [OK] All dependencies installed:
pip list | findstr /i "flask oracledb pillow werkzeug cryptography"

:: ── Run app ───────────────────────────────────────────────────
echo.
echo [4/4] Starting Flask app...
echo ═══════════════════════════════════════════════════════════
echo  App running at: http://localhost:5000
echo  Press CTRL+C to stop
echo ═══════════════════════════════════════════════════════════
echo.
mkdir temp_cache 2>nul
python tabulation_web.py
goto :end

:: ═══════════════════════════════════════════════════════════════════
:docker_mode
:: ═══════════════════════════════════════════════════════════════════
echo.
echo [MODE] Docker mode selected.
echo.

set IMAGE_NAME=nu-tabulation
set CONTAINER_NAME=nu-tabulation-app
set HOST_PORT=5000

:: ── Check Docker ──────────────────────────────────────────────
echo [1/5] Checking Docker...
docker --version >nul 2>&1
if %errorlevel% NEQ 0 (
    echo [!] Docker not found. Installing via winget...
    winget install -e --id Docker.DockerDesktop --silent --accept-package-agreements --ac
```

```
    if %errorlevel% NEQ 0 (
        echo [INFO] Downloading Docker Desktop installer...
        curl -L "https://desktop.docker.com/win/main/amd64/Docker%%20Desktop%%20Installer
        echo [INFO] Running installer (follow the GUI prompts)...
        "%TEMP%\DockerInstaller.exe" install --quiet
    )
    echo [INFO] Waiting 45s for Docker to start...
    timeout /t 45 /nobreak >nul
)

docker info >nul 2>&1
if %errorlevel% NEQ 0 (
    echo [!] Docker daemon not running. Starting Docker Desktop...
    start "" "C:\Program Files\Docker\Docker\Docker Desktop.exe"
    timeout /t 40 /nobreak >nul
    docker info >nul 2>&1
    if %errorlevel% NEQ 0 (
        echo [ERROR] Docker still not ready. Please start Docker Desktop manually and re-
        pause
        exit /b 1
    )
)
echo [OK] Docker is ready.
docker --version

:: ── Verify required files ──────────────────────────────────────
echo.
echo [2/5] Checking required files...
if not exist "Dockerfile" (
    echo [ERROR] Dockerfile not found in %PROJECT_DIR%
    echo         Make sure Dockerfile and requirements.txt are in the project root.
    pause
    exit /b 1
)
if not exist "requirements.txt" (
    echo [ERROR] requirements.txt not found.
    pause
    exit /b 1
)
echo [OK] Dockerfile and requirements.txt found.

:: ── Stop old container ─────────────────────────────────────────
echo.
echo [3/5] Removing old container...
docker stop %CONTAINER_NAME% >nul 2>&1
docker rm   %CONTAINER_NAME% >nul 2>&1
echo [OK] Done.

:: ── Build ──────────────────────────────────────────────────────
echo.
echo [4/5] Building Docker image (first time takes 2-5 min)...
docker build -t %IMAGE_NAME% .
if %errorlevel% NEQ 0 (
    echo [ERROR] Build failed. Check errors above.
    pause
    exit /b 1
```

```
)
echo [OK] Image built: %IMAGE_NAME%

:: ── Run ──────────────────────────────────────────────────────────
echo.
echo [5/5] Starting container...
docker run -d ^
    --name %CONTAINER_NAME% ^
    --restart unless-stopped ^
    -p %HOST_PORT%:5000 ^
    -v "%PROJECT_DIR%temp_cache:/app/temp_cache" ^
    %IMAGE_NAME%

if %errorlevel% NEQ 0 (
    echo [ERROR] Container failed to start.
    docker logs %CONTAINER_NAME%
    pause
    exit /b 1
)

echo.
echo ═══════════════════════════════════════════════════════════════
echo  SUCCESS!  App running at: http://localhost:%HOST_PORT%
echo ═══════════════════════════════════════════════════════════════
echo  docker logs -f %CONTAINER_NAME%   <- view live logs
echo  docker stop %CONTAINER_NAME%      <- stop app
echo  docker restart %CONTAINER_NAME%   <- restart
echo ═══════════════════════════════════════════════════════════════
timeout /t 3 /nobreak >nul
start http://localhost:%HOST_PORT%

:end
echo.
pause
```

`install_and_run.sh` — Linux / macOS

```
#!/bin/bash
# ═══════════════════════════════════════════════════════════════
#  NU Tabulation Archive  -  Linux/macOS Full Setup Script
#  Supports: Ubuntu, Debian, CentOS, RHEL, Fedora, macOS
# ═══════════════════════════════════════════════════════════════

set -e
PROJECT_DIR="$(cd "$(dirname "${BASH_SOURCE[^0]}")" && pwd)"
cd "$PROJECT_DIR"

GREEN='\033[0;32m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
CYAN='\033[0;36m'
NC='\033[0m'

echo ""
```

```
echo -e "${CYAN}═══════════════════════════════════════════════════════════${NC}"
echo -e "${CYAN}  NU TABULATION ARCHIVE  -  Linux/macOS Full Setup${NC}"
echo -e "${CYAN}═══════════════════════════════════════════════════════════${NC}"
echo ""
echo "Choose setup mode:"
echo "  [^1] Run with Python directly (install pip dependencies)"
echo "  [^2] Run with Docker          (recommended for deployment)"
echo ""
read -p "Enter 1 or 2: " CHOICE

# ═══════════════════════════════════════════════════════════════════
# DETECT OS
# ═══════════════════════════════════════════════════════════════════
detect_os() {
    if [[ "$OSTYPE" == "darwin"* ]]; then
        OS="macos"
    elif [ -f /etc/os-release ]; then
        . /etc/os-release
        case "$ID" in
            ubuntu|debian|linuxmint) OS="debian" ;;
            centos|rhel|rocky|almalinux) OS="rhel" ;;
            fedora) OS="fedora" ;;
            *) OS="unknown" ;;
        esac
    fi
    echo -e "${GREEN}[OK]${NC} Detected OS: $OS"
}

# ═══════════════════════════════════════════════════════════════════
if [ "$CHOICE" == "1" ]; then
# ═══════════════════════════════════════════════════════════════════
    echo ""
    echo -e "${CYAN}[MODE] Python direct mode${NC}"
    detect_os

    # ── Install Python 3.11 ──────────────────────────────────
    echo ""
    echo "[1/4] Checking Python..."
    if ! command -v python3 &>/dev/null; then
        echo "[!] Python3 not found. Installing..."
        if [ "$OS" == "debian" ]; then
            sudo apt-get update -qq
            sudo apt-get install -y python3.11 python3.11-venv python3-pip \
                                    libjpeg-turbo8 libpng-dev libfreetype6-dev
        elif [ "$OS" == "rhel" ]; then
            sudo dnf install -y python3.11 python3-pip libjpeg-turbo libpng freetype
        elif [ "$OS" == "fedora" ]; then
            sudo dnf install -y python3.11 python3-pip libjpeg libpng freetype
        elif [ "$OS" == "macos" ]; then
            if ! command -v brew &>/dev/null; then
                /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/ins
            fi
            brew install python@3.11 jpeg libpng freetype
        fi
    fi
    python3 --version
```

```
    echo -e "${GREEN}[OK]${NC} Python found."

    # ── Create venv ──────────────────────────────────────────────
    echo ""
    echo "[2/4] Setting up virtual environment..."
    if [ ! -f "$PROJECT_DIR/venv/bin/activate" ]; then
        python3 -m venv "$PROJECT_DIR/venv"
        echo -e "${GREEN}[OK]${NC} Virtual environment created."
    else
        echo -e "${GREEN}[OK]${NC} Virtual environment already exists."
    fi
    source "$PROJECT_DIR/venv/bin/activate"

    # ── Install pip packages ─────────────────────────────────────
    echo ""
    echo "[3/4] Installing Python dependencies..."
    pip install --upgrade pip -q
    pip install flask==3.1.3 \
                oracledb==3.4.2 \
                pillow==12.1.1 \
                werkzeug==3.1.5 \
                cryptography==46.0.5 \
                gunicorn==23.0.0
    echo ""
    echo -e "${GREEN}[OK]${NC} Installed packages:"
    pip list | grep -iE "flask|oracledb|pillow|werkzeug|cryptography|gunicorn"

    # ── Run ──────────────────────────────────────────────────────
    echo ""
    echo "[4/4] Starting Flask app..."
    mkdir -p temp_cache
    echo -e "${CYAN}═══════════════════════════════════════════════════════════════${NC}"
    echo -e "${CYAN}  App running at: http://localhost:5000${NC}"
    echo -e "${CYAN}  Press CTRL+C to stop${NC}"
    echo -e "${CYAN}═══════════════════════════════════════════════════════════════${NC}"
    python tabulation_web.py

# ═══════════════════════════════════════════════════════════════════
elif [ "$CHOICE" == "2" ]; then
# ═══════════════════════════════════════════════════════════════════
    echo ""
    echo -e "${CYAN}[MODE] Docker mode${NC}"
    detect_os

    IMAGE_NAME="nu-tabulation"
    CONTAINER_NAME="nu-tabulation-app"
    HOST_PORT=5000

    # ── Install Docker ───────────────────────────────────────────
    echo ""
    echo "[1/5] Checking Docker..."
    if ! command -v docker &>/dev/null; then
        echo "[!] Docker not found. Installing..."
        if [ "$OS" == "debian" ]; then
            sudo apt-get update -qq
            sudo apt-get install -y ca-certificates curl gnupg lsb-release
```

```
        sudo install -m 0755 -d /etc/apt/keyrings
        curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
            sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
        echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/doc
            https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
            sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
        sudo apt-get update -qq
        sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-
        sudo usermod -aG docker "$USER"
        sudo systemctl enable docker
        sudo systemctl start docker
    elif [ "$OS" == "rhel" ] || [ "$OS" == "fedora" ]; then
        sudo dnf -y install dnf-plugins-core
        sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/
        sudo dnf install -y docker-ce docker-ce-cli containerd.io
        sudo systemctl enable docker
        sudo systemctl start docker
        sudo usermod -aG docker "$USER"
    elif [ "$OS" == "macos" ]; then
        if ! command -v brew &>/dev/null; then
            /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/ins
        fi
        brew install --cask docker
        open /Applications/Docker.app
        echo "[INFO] Waiting 30s for Docker to start..."
        sleep 30
    fi
    echo -e "${GREEN}[OK]${NC} Docker installed."
fi

# Start docker daemon if not running
if ! docker info &>/dev/null 2>&1; then
    echo "[!] Docker daemon not running. Starting..."
    if [ "$OS" == "macos" ]; then
        open /Applications/Docker.app
        sleep 25
    else
        sudo systemctl start docker
    fi
fi
docker --version
echo -e "${GREEN}[OK]${NC} Docker is ready."

# ── Verify required files ──────────────────────────────────
echo ""
echo "[2/5] Checking required files..."
[ ! -f "Dockerfile" ]       && echo -e "${RED}[ERROR] Dockerfile not found!${NC}"
[ ! -f "requirements.txt" ] && echo -e "${RED}[ERROR] requirements.txt not found!${N
echo -e "${GREEN}[OK]${NC} Required files found."

# ── Remove old container ───────────────────────────────────
echo ""
echo "[3/5] Removing old container..."
docker stop "$CONTAINER_NAME" 2>/dev/null || true
docker rm   "$CONTAINER_NAME" 2>/dev/null || true
echo -e "${GREEN}[OK]${NC} Done."
```

```
    # ── Build ──────────────────────────────────────────────────────
    echo ""
    echo "[4/5] Building Docker image (first time: 2-5 min)..."
    docker build -t "$IMAGE_NAME" .
    echo -e "${GREEN}[OK]${NC} Image built: $IMAGE_NAME"

    # ── Run ────────────────────────────────────────────────────────
    echo ""
    echo "[5/5] Starting container..."
    mkdir -p "$PROJECT_DIR/temp_cache"
    docker run -d \
        --name "$CONTAINER_NAME" \
        --restart unless-stopped \
        -p "$HOST_PORT:5000" \
        -v "$PROJECT_DIR/temp_cache:/app/temp_cache" \
        "$IMAGE_NAME"

    echo ""
    echo -e "${GREEN}═══════════════════════════════════════════════════════════${NC}"
    echo -e "${GREEN}  SUCCESS! App running at: http://localhost:$HOST_PORT${NC}"
    echo -e "${GREEN}═══════════════════════════════════════════════════════════${NC}"
    echo "  docker logs -f $CONTAINER_NAME   <- live logs"
    echo "  docker stop $CONTAINER_NAME      <- stop"
    echo "  docker restart $CONTAINER_NAME   <- restart"
    echo -e "${GREEN}═══════════════════════════════════════════════════════════${NC}"
else
    echo -e "${RED}[ERROR] Invalid choice.${NC}"
    exit 1
fi
```

Make it executable on Linux/macOS:

```
chmod +x install_and_run.sh
./install_and_run.sh
```

Summary of Answers

Question	Answer
Run on Linux?	✅ Yes — Docker works great on Linux, use `.sh` script instead of `.bat` [1] [4]
Need Python/Flask before Docker?	❌ No — Docker installs everything inside the container automatically
Install script for direct Python run?	✅ Both scripts above handle Python + venv + all pip packages, or full Docker setup — your choice at runtime

⁂

1. https://docs.docker.com/desktop/setup/install/linux/
2. https://www.youtube.com/watch?v=OQV9E9uz8K4

3. https://stackoverflow.com/questions/12680998/run-bat-in-linux-environment/12681029
4. https://www.digitalocean.com/community/tutorials/how-to-install-and-use-docker-on-ubuntu-22-04
5. https://www.reddit.com/r/linux4noobs/comments/182w84b/how_to_run_a_bat_file_on_linux/
6. https://learn.microsoft.com/en-us/answers/questions/2671485/using-remote-shell-from-linux-to-run-bat-files-on
7. https://www.linux.org/threads/running-windows-batch-files-on-linux.11205/
8. https://linuxconfig.org/how-to-install-docker-desktop-on-ubuntu-26-04
9. https://www.docker.com/products/docker-desktop/
10. https://stackoverflow.com/questions/238013/invoking-windows-batch-file-from-linux
11. https://www.youtube.com/watch?v=fJLiHXcIBJQ
12. https://docs.docker.com/desktop/setup/install/linux/ubuntu/
13. https://www.docker.com/get-started/
14. https://learn.arm.com/install-guides/docker/docker-desktop-arm-linux/
15. https://docs.docker.com/desktop/setup/install/linux/archlinux/